

1

Introdução

Duas ideias brilham no veludo do joalheiro. O primeiro é o cálculo, o segundo, o algoritmo. O cálculo e o rico corpo da matemática a análise a que deu origem tornou a ciência moderna possível; mas tem sido o algoritmo que tornou possível o mundo moderno.

—David Berlinski, *The Advent of the Algorithm*, 2000

Por que você precisa estudar algoritmos? Se você vai ser um computador profissional, existem razões práticas e teóricas para estudar algoritmos. Do ponto de vista prático, você deve conhecer um conjunto padrão de importantes algoritmos de diferentes áreas da computação; além disso, você deve ser capaz de projetar novos algoritmos e analisar sua eficiência. Do ponto de vista teórico ponto, o estudo de algoritmos, às vezes chamados de **algoritmos**, passou a ser reconhecida como a pedra angular da ciência da computação. David Harel, em seu delicioso livro claramente intitulado *Algorithmics: the Spirit of Computing*, coloque-o da seguinte forma:

Algoritmia é mais do que um ramo da ciência da computação. É o núcleo de ciência da computação e, com toda a justiça, pode ser considerada relevante para a maioria dos ciência, negócios e tecnologia. [Har92, p. 6]

Mas mesmo que você não seja um estudante de um programa relacionado à computação, há razões convincentes para estudar algoritmos. Para ser franco, programas de computador não existiria sem algoritmos. E com os aplicativos de computador se tornando indispensável em quase todos os aspectos de nossa vida profissional e pessoal, estudando algoritmos se tornam uma necessidade para mais e mais pessoas.

Outra razão para estudar algoritmos é sua utilidade no desenvolvimento de habilidades alíticas. Afinal, os algoritmos podem ser vistos como tipos especiais de soluções para problemas - não apenas respostas, mas procedimentos precisamente definidos para obter respostas. Consequentemente, técnicas de design de algoritmo específico podem ser interpretadas como problema resolver estratégias que podem ser úteis independentemente de haver um computador envolvido. Claro, a precisão inerentemente imposta pelo pensamento algorítmico limita o tipos de problemas que podem ser resolvidos com um algoritmo. Você não vai encontrar, pois por exemplo, um algoritmo para viver uma vida feliz ou se tornar rico e famoso. Em

por outro lado, essa precisão exigida tem uma vantagem educacional. Donald Knuth, um dos cientistas da computação mais proeminentes na área algorítmica, coloque da seguinte forma:

Uma pessoa bem treinada em ciência da computação sabe como construí-los, manipulá-los, entendê-los, analisá-los. Este conhecimento é uma preparação para muito mais do que programas; é uma ferramenta mental de uso geral que será útil na compreensão de outros assuntos, sejam eles química, linguística ou música, etc. A razão para isso pode ser entendida da seguinte maneira. Muitas vezes se diz que uma pessoa não entende realmente algo até depois de ensiná-lo a outra pessoa. Na verdade, uma pessoa não entende algo até depois de ensiná-lo a um *computador*, ou seja, como um algoritmo. . . Uma tentativa de formalizar as coisas de uma maneira muito mais profunda do que se simplesmente a maneira tradicional. [Knu96, p. 9]

Consideramos a noção de algoritmo na Seção 1.1. Como exemplo de algoritmos para o mesmo problema: calcular o máximo divisor comum. Existem várias razões para esta escolha. Primeiro, ele lida com um problema que everybody de seus dias de escola secundária. Em segundo lugar, para o mesmo problema pode frequentemente ser resolvido por vários métodos. Os algoritmos diferem em sua ideia, nível de sofisticação e eficiência. Esses algoritmos merecem ser introduzidos primeiro, tanto por causa da popularidade no famoso tratado de Euclides há mais de dois mil anos e pela importância duradouros. Finalmente, a investigação desse problema leva a algumas observações gerais sobre várias propriedades importantes dos algoritmos em geral.

A seção 1.2 trata da solução de problemas algorítmicos. Lá discutimos questões importantes relacionadas com a concepção e análise de algoritmos. Aspectos da resolução de problemas algorítmicos vão desde a análise de um algoritmo para estabelecer sua correção e a eficiência. A seção não contém uma receita mágica para projetar um algoritmo para um problema arbitrário. É um fato bem estabelecido que tal coisa não existe. Ainda assim, o material da Seção 1.2 deve ser útil para o projeto na concepção e análise de algoritmos.

A seção 1.3 é dedicada a alguns tipos de problemas que são extremamente importantes para o estudo de algoritmos e suas aplicações. São livros didáticos (por exemplo, [Sed11]) organizados em torno de uma visão - compartilhada por muitos outros - que uma organização baseada em técnicas é superior. Em qualquer caso, é muito importante estar ciente de alguns tipos de problemas amigos. Não são apenas os problemas mais comuns em tipos de aplicativos da vida real, eles são usados ao longo do livro para técnicas específicas de projeto de algoritmos.

A Seção 1.4 contém uma revisão das estruturas de dados e como elas servem como uma referência em vez de uma discussão deliberada e

uma exposição mais detalhada, há uma abundância de bons livros sobre o assunto, a maioria deles sob medida para uma linguagem de programação específica.

1.1 O que é um algoritmo?

Embora não haja uma redação universalmente aceita para descrever esta noção, há é um acordo geral sobre o que o conceito significa:

Um **algoritmo** é uma sequência de instruções inequívocas para resolver um problema, ou seja, para obter uma saída necessária para qualquer entrada legítima em uma quantidade finita de tempo.

Essa definição pode ser ilustrada por um diagrama simples (Figura 1.1).
A referência a "instruções" na definição implica que há algum coisa ou alguém capaz de compreender e seguir as instruções dadas. Chamamos isso de "computador", tendo em mente que antes do computador eletrônico foi inventado, a palavra "computador" significava um ser humano envolvido em cálculos numéricos. Hoje em dia, é claro, "computadores" são os onipresentes aparelhos eletrônicos que se tornaram indispensáveis em quase tudo o que fazemos. Observe, no entanto, que embora a maioria dos algoritmos sejam de fato destinados a eventual implementação de computador, a noção de algoritmo não depende de tal suposição.

Como exemplos que ilustram a noção do algoritmo, consideramos neste seção três métodos para resolver o mesmo problema: computar o maior divisor comum de dois inteiros. Esses exemplos nos ajudarão a ilustrar vários pontos importantes:

- O requisito de não ambiguidade para cada etapa de um algoritmo não pode ser comprometido.
- O intervalo de entradas para o qual um algoritmo funciona deve ser especificado com cuidado.
- O mesmo algoritmo pode ser representado de várias maneiras diferentes.
- Podem existir vários algoritmos para resolver o mesmo problema.

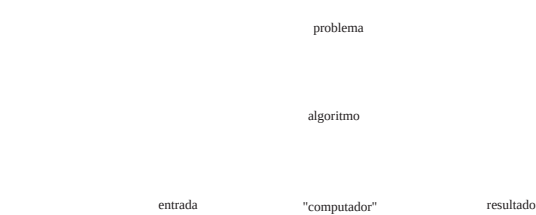


FIGURA 1.1 A noção do algoritmo.

Algoritmos para o mesmo problema podem ser baseados em
pode resolver o problema com velocidades drasticamente di

Lembre-se de que o maior divisor comum de dois não negat
inteiros m e n , denotados $\gcd(m, n)$, são definidos como o maio
ambos m e n uniformemente, ou seja, com um resto de zero. Eucl
século AC) delineou um algoritmo para resolver este problema e
de seus *elementos* mais famosos por sua exposição sistemática da
termos, **o algoritmo de Euclides** é baseado na aplicação repetida

$$\text{mdc}(m, n) = \text{mdc}(n, m \bmod n)$$

onde $m \bmod n$ é o resto da divisão de m por n , até que $m \bmod n$ s
a 0. Como $\text{mdc}(m, 0) = m$ (por quê?), o último valor de m també
divisor do m inicial e n .

Por exemplo, $\gcd(60, 24)$ pode ser calculado da seguinte

$$\text{mdc}(60, 24) = \text{mdc}(24, 12) = \text{mdc}($$

(Se você não está impressionado com este algoritmo, tente encon
de números maiores, como aqueles no Problema 6 nos exercícios

Aqui está uma descrição mais estruturada deste algoritmo:

Algoritmo de Euclides para calcular $\gcd(m, n)$

Etapa 1 Se $n = 0$, retorne o valor de m como a resposta e pa
prossiga para a Etapa 2.

Etapa 2 Divida m por n e atribua o valor do resto a r .

Etapa 3 Atribua o valor de n a m e o valor de r a n . Vá para

Como alternativa, podemos expressar o mesmo algoritmo em

ALGORITMO *Euclides* (m, n)

```
// Calcula  $\gcd(m, n)$  pelo algoritmo de Euclides
// Entrada: Dois inteiros não negativos, não ambos zero,  $m \in \mathbb{N}$  e  $n \in \mathbb{N}$ 
// Saída: Máximo divisor comum de  $m$  e  $n$ 
enquanto  $n \neq 0$  faz
     $r \leftarrow m \bmod n$ 
     $m \leftarrow n$ 
     $n \leftarrow r$ 
retornar  $m$ 
```

Como sabemos que o algoritmo de Euclides finalmente che
segue da observação de que o segundo inteiro do par fica menor c
cada iteração e não pode se tornar negativo. Na verdade, o novo v
a próxima iteração é $m \bmod n$, que é sempre menor que n (por qu
do segundo inteiro eventualmente torna-se 0 e o algoritmo para.

Assim como com muitos outros problemas, existem vá o maior divisor comum. Vejamos os outros dois métodos pa lem. O primeiro é simplesmente baseado na definição do m m e n como o maior número inteiro que divide os dois núme um divisor comum não pode ser maior do que o menor dess denotará por $t = \min \{ m, n \}$. Portanto, podemos começar v m e n : em caso affirmative , t é a resposta; se não, simplesm tente novamente. (Como sabemos se o processo acabará par para os números 60 e 24, o algoritmo tentará primeiro 24, de chega a 12, onde para.

Algoritmo de verificação de número inteiro consecutivo j

- Etapa 1** Atribua o valor de $\min \{ m, n \}$ a t .
- Etapa 2** Divida m por t . Se o restante desta divisão for caso contrário, vá para a Etapa 4.
- Etapa 3** Divida n por t . Se o restante desta divisão for t como a resposta e parada; caso contrário, v
- Etapa 4** Diminua o valor de t em 1. Vá para a Etapa 2.

Observe que, ao contrário do algoritmo de Euclides, es não funciona corretamente quando um de seus números de e ilustra porque é tão importante especificar o conjunto de ent e com cuidado.

O terceiro procedimento para encontrar o maior divisoiar para você do ensino médio.

Procedimento do ensino médio para computação $\gcd (m, n$

- Etapa 1** Encontre os fatores primos de m .
- Etapa 2** Encontre os fatores primos de n .
- Etapa 3** Identifique todos os fatores comuns nas duas Etapa 1 e Etapa 2. (Se p for um fator comum em m e n , respectivamente, deve ser repetido
- Etapa 4** Calcule o produto de todos os fatores comuns maior divisor comum dos números dados.

Assim, para os números 60 e 24, obtemos

$$\begin{aligned} 60 &= 2 \cdot 2 \cdot 3 \cdot 5 \\ 24 &= 2 \cdot 2 \cdot 2 \cdot 3 \\ \text{mdc} (60 , 24) &= 2 \cdot 2 \cdot 3 \end{aligned}$$

A saudade dos dias em que aprendemos este método n de notar que o último procedimento é muito mais complexo algoritmo. (Vamos discutir os métodos para encontrar e com de algoritmos no próximo capítulo.) Além de reduzir a eficii o procedimento escolar não se qualifica, na forma apresenta Por quê? Porque as etapas principais de fatoração não são de

exigem uma lista de números primos, e eu fortemente suspeito que o professor de matemática não explicou como obter essa lista. Isso é de picuinhas desnecessárias. A menos que esse problema seja resolvido, não há programa de implementação deste procedimento. Aliás, a Etapa 3 é claramente o suficiente. Sua ambigüidade é muito mais fácil de corrigir em etapas, no entanto. Como você encontraria elementos comuns em duas listas? Então, vamos introduzir um algoritmo simples para gerar primos não excedendo qualquer número inteiro $n > 1$. Provavelmente foi inventado e é conhecida como a **peneira de Eratóstenes** (ca. 200 aC). O algoritmo inicializa uma lista de candidatos principais com inteiros consecutivos. Em sua primeira iteração, o algoritmo elimina da lista todos os múltiplos de 2 e assim por diante. Em seguida, ele passa para o próximo item e elimina seus múltiplos. (Nesta versão simples, há uma sobrecarga porque múltiplos de 2, como 6, são eliminados mais de uma vez.) Nenhuma passagem é necessária: uma vez que o próprio 4 e todos os seus múltiplos também são eliminados em um passe anterior. O próximo número restante na lista, usado na terceira passagem, é 5. O algoritmo continua desta maneira até que os números podem ser eliminados da lista. Os restantes inteiros são os primos necessários.

Como exemplo, considere a aplicação do algoritmo para encontrar primos não excedendo $n = 25$:

2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
2	3			5		7			9				11				13				15		
2	3			5		7							11				13						
2	3			5		7							11				13						

Para este exemplo, não são necessárias mais passagens porque todos os números já eliminados em iterações anteriores do algoritmo. Os restantes os números na lista são os primos consecutivos menores ou iguais a 25. Qual é o maior número p cujos múltiplos ainda podem permanecer? Para fazer novas iterações do algoritmo necessárias? Antes de responder a esta questão, vamos primeiro notar que se p é um número cujos múltiplos não foram eliminados na passagem atual, então o primeiro múltiplo que devemos considerar é $2p$. Os múltiplos menores $2p, \dots, (p - 1)p$ foram eliminados em passagens anteriores. Essa observação ajuda a evitar a eliminação do mesmo número mais de uma vez. Obviamente, pp não deve ser considerado, pois p é o maior número considerado e, portanto, p não é arredondado para baixo (denotado $\lfloor n \rfloor$ usando a chamada função piso). No seguinte pseudocódigo que existe uma função disponível para verificar se um número é primo, poderíamos verificar a desigualdade $pp \leq n$ com a condição lá.

// Implementa a peneira de Eratóstenes

// Input: Um inteiro positivo $n > 1$ // Saída: Array L de todos os números primos menores ou ig

```

para  $p \leftarrow 2$  para  $q$  fazer  $Um[p] \leftarrow p$ 
para  $p \leftarrow 2$  a  $n$  do // veja a nota antes do pseudocódigo
    se  $A[p] = 0$  //  $p$  não foi eliminado nos passes anteriores
         $j \leftarrow p * p$ 
        enquanto  $j \leq n$  faça
             $A[j] \leftarrow 0$  // marca o elemento como eliminado
             $j \leftarrow j + p$ 
// copia os elementos restantes de  $A$  para a matriz  $L$  dos primos
 $i \leftarrow 0$ 
para  $p \leftarrow 2$  para  $n$  fazer
    se  $A[p] = 0$ 
         $L[i] \leftarrow A[p]$ 
         $i \leftarrow i + 1$ 
retornar  $L$ 

```

Então, agora podemos incorporar a peneira de Eratóstenes ao ensino médio procedimento para obter um algoritmo legítimo para calcular a maior divisão comum sor de dois inteiros positivos. Observe que um cuidado especial deve ser exercido se um ou ambos os números de entrada são iguais a 1: porque os matemáticos não consideram 1 para ser um número primo, estritamente falando, o método não funciona para essas entradas.

Antes de sairmos desta seção, mais um comentário é necessário. O exame- apesar dos aspectos considerados nesta seção, a maioria dos algoritmos em uso hoje - mesmo aqueles que são implementados como programas de computador - não lidam com problemas matemáticos. Procure algoritmos que nos ajudem em nosso diário rotinas, profissionais e pessoais. Que esta onipresença de algoritmos em para- O mundo do dia fortalece sua determinação de aprender mais sobre esses motores fascinantes da era da informação.

Exercícios 1.1

- Faça algumas pesquisas sobre al-Khorezmi (também al-Khwarizmi), o homem de quem nome, a palavra “algoritmo” é derivada. Em particular, você deve aprender o que as origens das palavras “algoritmo” e “álgebra” têm em comum.
- Dado que o objetivo oficial do sistema de patentes dos EUA é a promoção das “artes úteis”, você acha que os algoritmos são patenteáveis neste país? Eles deveriam ser?
- Anote as instruções de direção para ir da escola até sua casa com a precisão exigida pela descrição de um algoritmo.
 - Escreva uma receita para cozinhar seu prato favorito com a precisão exigido por um algoritmo.
- Projete um algoritmo para computação $\lfloor \sqrt{\quad} \rfloor$ n para qualquer número inteiro positivo n . Além de

5. Projete um algoritmo para encontrar todos os elementos com números. Por exemplo, para as listas 2, 5, 5, 5 e 2, 3, 5, 5 deve ser 2, 5, 5. Qual é o número máximo de comparações que faz se os comprimentos das duas listas fornecidas são m e n .
6. **a.** Encontre $\text{mdc}(31415, 14142)$ aplicando o algoritmo de Euclides.
B. Estime quantas vezes mais rápido será encontrar o $\text{mdc}(m, n)$ usando o algoritmo de Euclides comparado com o algoritmo baseado em divisões sucessivas de $\min\{m, n\}$ até $\text{mdc}(m, n)$.
7. Prove a igualdade $\text{mdc}(m, n) = \text{mdc}(n, m \bmod n)$ para cada par de inteiros m e n .
8. O que o algoritmo de Euclides faz para um par de inteiros em menor que o segundo? Qual é o número máximo de vezes que acontece durante a execução do algoritmo em tal entrada?
9. **a.** Qual é o número mínimo de divisões feitas pelo algoritmo entre todas as entradas $1 \leq m, n \leq 10$?
B. Qual é o número máximo de divisões feitas pelo algoritmo entre todas as entradas $1 \leq m, n \leq 10$?
10. **a.** O algoritmo de Euclides, conforme apresentado no tratado em vez de divisões inteiras. Escreva pseudocódigo para o algoritmo.
B. O jogo de Euclides (ver [Bog]) começa com dois inteiros a e b . Dois jogadores passam na vez. Em cada movimento, o jogador remove um número positivo igual à diferença de dois números no quadro; este número deve ser novo, ou seja, diferente de zero. O jogador que não consegue se mover perde. Você escolhe se mover primeiro ou segundo neste jogo?
11. O algoritmo de Euclides estendido determina não apenas o maior divisor comum d de dois inteiros positivos m e n , mas também inteiros x e y , de modo que $mx + ny = d$.
O. Procure uma descrição do algoritmo de Euclides estendido (ver [P. 13]) e implemente-o no idioma de sua escolha.
B. Modifique seu programa para encontrar soluções inteiras para a equação $ax + by = c$ com qualquer conjunto de coeficientes inteiros.
12. *Portas decabinets* Existem n armários em um corredor, numerados de 1 a n . Inicialmente, todas as portas dos armários estão fechadas. Você começa no armário # 1. Na i -ésima passagem, você alterna a porta de cada i th armário: se a porta estiver fechada, você a abre; se estiver aberta, você a fecha. Após a última passagem, quais portas estão fechadas? Quantos deles estão abertos?

1.2 Fundamentos da Resolução de Problemas Algorítmicos

Começemos reiterando um ponto importante feito na introdução a este capítulo:

Podemos considerar os algoritmos como soluções procedimentais para problemas.

Essas soluções não são respostas, mas instruções específicas para obter respostas. Isto é esta ênfase em procedimentos construtivos precisamente definidos que torna o computador ciência distinta de outras disciplinas. Em particular, isso o distingue da matemática orética, cujos praticantes normalmente ficam satisfeitos em apenas provar a existência de uma solução para um problema e, possivelmente, investigando a solução propriedades.

Agora listamos e discutimos brevemente uma sequência de etapas que normalmente percorremos no projeto e análise de um algoritmo (Figura 1.2).

Compreendendo o problema

De uma perspectiva prática, a primeira coisa que você precisa fazer antes de projetar um algoritmo é entender completamente o problema dado. Leia o problema descreva cuidadosamente e faça perguntas se tiver alguma dúvida sobre o problema, faça alguns pequenos exemplos à mão, pense em casos especiais e faça perguntas novamente se necessário.

Existem alguns tipos de problemas que surgem em aplicativos de computação bastante frequentemente. Nós os revisamos na próxima seção. Se o problema em questão for um dos eles, você pode ser capaz de usar um algoritmo conhecido para resolvê-lo. Claro que ajuda a entender como esse algoritmo funciona e a conhecer seus pontos fortes e pontos fracos, especialmente se você tiver que escolher entre vários algoritmos disponíveis. Mas muitas vezes você não encontrará um algoritmo prontamente disponível e terá que projetar seu próprio. A sequência de etapas descritas nesta seção deve ajudá-lo nesta tarefa emocionante, mas nem sempre fácil.

Uma entrada para um algoritmo especifica uma *instância* do problema do algoritmo resolve. É muito importante especificar exatamente o conjunto de instâncias do algoritmo precisa lidar. (Por exemplo, lembre-se das variações no conjunto de instâncias para os três algoritmos do maior divisor comum discutidos na seção anterior.) Se você não conseguir fazer isso, seu algoritmo pode funcionar corretamente para a maioria das entradas mas travar em algum valor “limite”. Lembre-se de que um algoritmo correto não é que na maioria das vezes funciona, mas um que funcione corretamente para *todas* legítimas entradas.

Não economize nesta primeira etapa do processo algorítmico de solução de problemas; caso contrário, você correrá o risco de retrabalho desnecessário.

Verificar as capacidades do dispositivo computacional

Depois de entender completamente um problema, você precisa verificar os recursos do dispositivo computacional ao qual o algoritmo se destina. A grande maioria de

Entenda o problema

Decide sobre:
meios computacionais,
resolução exata vs. resolução aproxi-
mada, técnica de design de algoritmo

Projete um algoritmo

Prove correção

Analise o algoritmo

Codifique o algoritmo

FIGURA 1.2 Projeto de algoritmo e processo de análise.

algoritmos em uso hoje ainda estão destinados a ser programados semelhante à máquina de von Neumann - uma arquitetura de computadores desenvolvida pelo proeminente matemático húngaro-americano John von Neuman (1903-1957), em colaboração com A. Burks e H. Goldstine, em 1946. A esta arquitetura é capturada pela chamada **máquina de acesso aleatório**. Seu pressuposto central é que as instruções são executadas uma a uma, uma operação de cada vez. Consequentemente, algoritmos projetados para serem executados em máquinas são chamados de **algoritmos sequenciais**.

O pressuposto central do modelo de RAM não se aplica a computadores que podem realizar operações simultaneamente, ou seja, que tiram vantagem dessa capacidade são chamados de **algoritmos paralelos**. As técnicas clássicas para projeto e análise de algoritmos sob o modelo de von Neumann permanecem a pedra angular da algorítmica para o futuro previsível.

Você deve se preocupar com a velocidade e a quantidade de memória de um computador em sua disposição? Se você está projetando um algoritmo como um exercício científico, a resposta é um não qualificado. Como você verá na Seção 2.1, a maioria dos cientistas da computação prefere estudar algoritmos em termos independentes de parâmetros de especificação para um determinado computador. Se você está projetando um algoritmo como uma ferramenta prática, a resposta pode depender de um problema que você precisa resolver. Mesmo os computadores "lentos" de hoje são quase inimaginavelmente rápidos. Consequentemente, em muitas situações, você não precisa se preocupar sobre um computador lento demais para a tarefa. Existem problemas importantes, no entanto, que são muito complexos por natureza ou precisam processar grandes volumes de dados ou lidar com aplicativos em que o tempo é crítico. Em tais situações, é imperativo estar ciente da velocidade e da memória disponível em um determinado sistema de computador.

Escolhendo entre a resolução de problemas exata e aproximada

A próxima decisão principal é escolher entre resolver o problema exatamente ou resolvendo-o aproximadamente. No primeiro caso, um algoritmo é chamado de *algoritmo exato* ; no último caso, um algoritmo é denominado *algoritmo de aproximação* . Por que optaria por um algoritmo de aproximação? Primeiro, existem problemas importantes que simplesmente não podem ser resolvidos exatamente na maioria das instâncias; exemplos incluem extrair raízes quadradas, resolver equações não lineares e avaliar integrais indefinidas. Em segundo lugar, os algoritmos disponíveis para resolver um problema exatamente podem ser inaceitavelmente lentos devido à complexidade intrínseca do problema. Isso acontece, em particular, para muitos problemas envolvendo um grande número de escolhas; você irá ver exemplos de tais problemas difíceis nos Capítulos 3, 11 e 12. Terceiro, um algoritmo de aproximação pode ser parte de um algoritmo mais sofisticado que resolve um problema exatamente.

Técnicas de Design de Algoritmo

Agora, com todos os componentes da solução de problemas algorítmicos no lugar, como você projeta um algoritmo para resolver um determinado problema? Esta é a principal questão. O livro procura responder ensinando-lhe várias técnicas gerais de design.

O que é uma técnica de projeto de algoritmo?

Uma *técnica de design de algoritmo* (ou "estratégia" ou "paradigma") é uma abordagem para resolver problemas de forma algorítmica que é aplicável a uma variedade de problemas de diferentes áreas da computação.

Verifique o índice deste livro e você verá que a maioria de seus capítulos são dedicados a técnicas de design individual. Eles destilam algumas ideias-chave que provaram ser úteis na criação de algoritmos. Aprendendo essas técnicas é de extrema importância pelas seguintes razões.

Primeiro, eles fornecem orientação para projetar algoritmos para novos problemas, ou seja, problemas para os quais não há algoritmo satisfatório conhecido. Portanto - para usar a linguagem de um famoso provérbio - aprender tais técnicas é semelhante a aprender

pescar em vez de receber um peixe capturado por outra pessoa. Não é verdade, de claro, que cada uma dessas técnicas gerais será necessariamente aplicável a todos os problemas que você pode encontrar. Mas, tomados em conjunto, eles constituem um poderoso conjunto de ferramentas que você achará bastante úteis em seus estudos e trabalho.

Em segundo lugar, os algoritmos são a base da ciência da computação. Toda ciência é interessada em classificar seu assunto principal, e a ciência da computação não é exceção. As técnicas de design de algoritmos tornam possível classificar algoritmos de acordo com a ideia de design subjacente; portanto, eles podem servir como uma forma natural para ambos

categorizar e estudar algoritmos.

Projetando um Algoritmo e Estruturas de Dados

Embora as técnicas de design de algoritmo forneçam um poderoso conjunto de aplicativos gerais aborda a resolução de problemas algorítmicos, projetando um algoritmo para um determinado problema ainda pode ser uma tarefa desafiadora. Algumas técnicas de design podem ser simplesmente inaplicável ao problema em questão. Às vezes, várias técnicas precisam ser combinados, e há algoritmos que são difíceis de identificar como aplicativos das técnicas de design conhecidas. Mesmo quando uma técnica de design particular é apropriada aplicável, obter um algoritmo muitas vezes requer uma engenhosidade não trivial por parte de o designer do algoritmo. Com a prática, ambas as tarefas - escolher entre as gerais técnicas e aplicá-las - fica mais fácil, mas raramente são fáceis.

Claro, deve-se prestar muita atenção na escolha de estruturas de dados adequadas apropriado para as operações realizadas pelo algoritmo. Por exemplo, a peneira de Eratóstenes introduzido na Seção 1.1 duraria mais se usássemos uma lista vinculada em vez de uma matriz em sua implementação (por quê?). Observe também que alguns dos técnicas de design de goritmo discutidas nos Capítulos 6 e 7 dependem intimamente de estruturar ou reestruturar dados especificando a instância de um problema. Há muitos anos, um livro influente proclamou a importância fundamental de ambos ritmos e estruturas de dados para programação de computadores por seu próprio título: *Algoritmos + Estruturas de dados = Programas* [Wir76]. No novo mundo da orientação a objetos gramática, as estruturas de dados permanecem crucialmente importantes para design e análise de algoritmos. Revisamos as estruturas de dados básicas na Seção 1.4.

Métodos de especificação de um algoritmo

Depois de projetar um algoritmo, você precisa especificá-lo de alguma forma. No Seção 1.1, para dar um exemplo, o algoritmo de Euclides é descrito em palavras (em um livre e também passo a passo) e em pseudocódigo. Estas são as duas opções que são mais amplamente usados hoje em dia para especificar algoritmos.

Usar uma linguagem natural tem um apelo óbvio; no entanto, o ambi- a identidade de qualquer linguagem natural faz uma descrição sucinta e clara dos algoritmos surpreendentemente difícil. No entanto, ser capaz de fazer isso é uma habilidade importante que você deve se esforçar para desenvolver no processo de aprendizagem de algoritmos.

Pseudocódigo é uma mistura de linguagem natural e linguagem de programação como construções. O pseudocódigo é geralmente mais necessário do que a linguagem natural, e seu

o uso geralmente produz descrições de algoritmo mais sucintas. Surpreendentemente, computador os cientistas nunca concordaram com uma única forma de pseudocódigo, deixando o livro didático autores com a necessidade de projetar seus próprios “dialetos”. Felizmente, esses dialetos são tão próximos uns dos outros que qualquer pessoa familiarizada com uma linguagem de programação moderna deve ser capaz de compreendê-los todos.

O dialeto deste livro foi selecionado para causar o mínimo de dificuldade para o leitor. Para por uma questão de simplicidade, omitimos declarações de variáveis e usamos recuo para mostrar o escopo de tais declarações como **for**, **if** e **while**. Como você viu no anterior seção, usamos uma seta “**--**” para a operação de atribuição e duas barras “**//**” para comentários.

Nos primeiros dias da computação, o veículo dominante para especificar algo- ritmos era um *fluxograma*, um método de expressar um algoritmo por uma coleção de formas geométricas conectadas contendo descrições das etapas do algoritmo. este técnica de representação provou ser inconveniente para todos, mas muito simples algoritmos; hoje em dia, ele só pode ser encontrado em livros antigos de algoritmos.

O estado da arte da computação ainda não atingiu um ponto em que um a descrição do algoritmo - seja em uma linguagem natural ou pseudocódigo - pode ser alimentada em um computador eletrônico diretamente. Em vez disso, ele precisa ser convertido em um programa de computador escrito em uma linguagem de computador específica. Podemos olhar para tais um programa como mais uma forma de especificar o algoritmo, embora seja preferível considerá-lo como a implementação do algoritmo.

Provando a Correção de um Algoritmo

Uma vez que um algoritmo tenha sido especificado, você deve provar sua *exatidão*. Isso é, você tem que provar que o algoritmo produz um resultado necessário para cada entrada em uma quantidade finita de tempo. Por exemplo, a correção do algoritmo de Euclides para calcular o maior divisor comum decorre da correção do igualdade $\text{mdc}(m, n) = \text{mdc}(n, m \bmod n)$ (que, por sua vez, precisa de uma prova; consulte o problema 7 nos Exercícios 1.1), a simples observação de que o segundo inteiro fica menor em cada iteração do algoritmo, e o fato de que o algoritmo para quando o o segundo inteiro torna-se 0.

Para alguns algoritmos, uma prova de correção é bastante fácil; para outros, pode ser bem complexo. A técnica comum para provar a correção é usar matemati- indução cal, porque as iterações de um algoritmo fornecem uma sequência natural de etapas necessário para tais provas. Pode valer a pena mencionar que, embora rastreie o o desempenho do algoritmo para algumas entradas específicas pode ser uma atividade muito interessante, não pode provar a exatidão do algoritmo de forma conclusiva. Mas para mostrar que um algoritmo está incorreto, você precisa de apenas uma instância de sua entrada para a qual o

algoritmo falha.

A noção de correção para algoritmos de aproximação é menos direta do que para algoritmos exatos. Para um algoritmo de aproximação, normalmente gostaríamos gostaria de poder mostrar que o erro produzido pelo algoritmo não excede um limite predefinido. Você pode encontrar exemplos de tais investigações no Capítulo 12.

Analisando um Algoritmo

Geralmente queremos que nossos algoritmos possuam várias qualidades. Depois da correção, de longe, o mais importante é a **eficiência**. Na verdade, existem dois tipos de algoritmo eficiência: eficiência de **tempo**, indicando o quão rápido o algoritmo é executado, e **espaço eficiência**, indicando quanta memória extra ele usa. Uma estrutura geral e técnicas específicas para analisar a eficiência de um algoritmo aparecem no Capítulo 2.

Outra característica desejável de um algoritmo é a **simplicidade**. Ao contrário de eficiência, que pode ser definida com precisão e investigada com rigor matemático, a simplicidade, como a beleza, está em um grau considerável nos olhos de quem vê. Para exemplo, a maioria das pessoas concordaria que o algoritmo de Euclides é mais simples do que o procedimento de ensino médio para computar $\text{mdc}(m, n)$, mas não está claro se eu- O algoritmo do clid é mais simples do que o algoritmo de verificação de inteiros consecutivos. Ainda, a simplicidade é uma característica importante do algoritmo a ser buscada. Por quê? Porque sim- algoritmos mais simples são mais fáceis de entender e programar; conseqüentemente, o os programas resultantes geralmente contêm menos bugs. Há também o inegável aes- apelo tético da simplicidade. Às vezes, algoritmos mais simples também são mais eficientes do que alternativas mais complicadas. Infelizmente, nem sempre é verdade, em que caso um acordo criterioso precisa ser feito.

Ainda outra característica desejável de um algoritmo é a **generalidade**. Tem, na verdade, duas questões aqui: generalidade do problema que o algoritmo resolve e o conjunto de entradas que aceita. Sobre o primeiro problema, observe que às vezes é mais fácil projetar um algoritmo para um problema apresentado em termos mais gerais. Considere, por exemplo, o problema de determinar se dois inteiros são relativamente primos, ou seja, se seu único divisor comum é igual a 1. É mais fácil projetar um algoritmo para um problema mais geral de cálculo do maior divisor comum de dois inteiros e, para resolver o primeiro problema, verifique se o mdc é 1 ou não. Existem situações, no entanto, em que projetar um algoritmo mais geral é desnecessário ou difícil ou mesmo impossível. Por exemplo, não é necessário classificar uma lista de n números para encontrar sua mediana, que é seu $\lceil n/2 \rceil$ º menor elemento. Dar outro exemplo, a fórmula padrão para raízes de uma equação quadrática não pode ser generalizado para lidar com polinômios de graus arbitrários.

Quanto ao conjunto de entradas, sua principal preocupação deve ser projetar um algoritmo que pode lidar com um conjunto de entradas que é natural para o problema em questão. Por exemplo, excluindo inteiros iguais a 1 como entradas possíveis para o maior divisor comum algoritmo seria bastante anormal. Por outro lado, embora o padrão fórmula para as raízes de uma equação quadrática vale para coeficientes complexos, nós normalmente não o implementaria neste nível de generalidade, a menos que esta capacidade é explicitamente necessário.

Se você não estiver satisfeito com a eficiência, simplicidade ou generalidade do algoritmo, você deve retornar à prancheta e redesenhar o algoritmo. Na verdade, mesmo que sua avaliação é positiva, ainda vale a pena buscar outras soluções algorítmicas. Lembre-se dos três algoritmos diferentes na seção anterior para calcular o maior divisor comum: geralmente, você não deve esperar obter o melhor algoritmo na primeira tentativa. No mínimo, você deve tentar ajustar o algoritmo que você

já tem. Por exemplo, fizemos várias melhorias em nossa implementação da peneira de Eratóstenes em comparação com seu contorno inicial na Seção 1.1. (Lata você se sairá bem se tiver em mente a seguinte observação de Antoine de Saint-Exupéry, o escritor, piloto e projetista de aeronaves francês: “A designer sabe que ele atingiu a perfeição não quando não há mais nada para adicionar, mas quando não houver mais nada para tirar.” ¹

Codificando um Algoritmo

A maioria dos algoritmos são destinados a serem implementados em última instância como programas de computador gramas. A programação de um algoritmo apresenta um perfil e uma oportunidade. o o perigo reside na possibilidade de fazer a transição de um algoritmo para um pro- grama incorretamente ou muito ineficiente. Alguns cientistas da computação influentes acredito fortemente que, a menos que a correção de um programa de computador seja comprovada com total rigor matemático, o programa não pode ser considerado correto. Eles desenvolveram técnicas especiais para fazer tais provas (ver [Gri81]), mas o

poder dessas técnicas de verificação formal é limitado até agora a muito pequeno programas.

Na prática, a validade dos programas ainda é estabelecida por meio de testes. Testar programas de computador é uma arte e não uma ciência, mas isso não significa que não há nada para aprender. Procure livros dedicados a testes e depuração; ainda mais importante, teste e depure seu programa completamente sempre que você implementar um algoritmo.

Observe também que ao longo do livro, assumimos que as entradas para algoritmos pertencem aos conjuntos especificados e, portanto, não requerem verificação. Ao implementar algoritmos como programas a serem usados em aplicativos reais, você deve fornecer tais verificações.

Claro, implementar um algoritmo corretamente é necessário, mas não suficiente: você não gostaria de diminuir o poder do seu algoritmo por um implemento ineficiente tação. Compiladores modernos fornecem uma certa rede de segurança a esse respeito, especialmente quando eles são usados em seu modo de otimização de código. Ainda assim, você precisa estar ciente de truques padrão como calcular o invariante de um loop (uma expressão que não altere seu valor) fora do loop, coletando subexpressões comuns, substituindo transferir operações caras por outras baratas, e assim por diante. (Veja [Ker99] e [Ben00] para uma boa discussão sobre ajuste de código e outras questões relacionadas ao programa de algoritmo ming.) Normalmente, essas melhorias podem acelerar um programa apenas por uma constante fator, enquanto um algoritmo melhor pode fazer a diferença no tempo de execução por pedidos de magnitude. Mas, uma vez que um algoritmo é selecionado, um aumento de velocidade de 10-50% pode valer a pena um esforço.

1 Encontrei essa necessidade de simplicidade de design em uma coleção de ensaios de Jon Bentley [Ben00]; o negócio de ensaios com uma variedade de problemas no design e implementação de algoritmos e são justificadamente intitulados *Programação Pérolas*. Recomendo sinceramente os escritos de Jon Bentley e Antoine de Saint-Exupéry.

Um programa de trabalho oferece uma oportunidade adicional de análise crítica do algoritmo subjacente. Essa análise é baseada no programa em várias entradas e, em seguida, analisando os resultados para discutir as vantagens e desvantagens desta abordagem para análise na Seção 2.6.

Concluindo, vamos enfatizar novamente a lição principal da Figura 1.2:

Como regra, um bom algoritmo é o resultado de esforços

Mesmo que você tenha a sorte de ter uma ideia algorítmica que parece perfeita, você ainda deve tentar ver se pode ser melhorado.

Na verdade, esta é uma boa notícia, pois torna o resultado final agradável. (Sim, pensei em chamar este livro de *The Joy of Algorithms*. Por outro lado, como saber quando parar? No mundo real, mais finem o cronograma de um projeto ou a impaciência de seu chefe o deveria ser: a perfeição é cara e, na verdade, nem sempre é necessária: um algoritmo é uma atividade semelhante à engenharia que exige objetivos concorrentes sob as restrições dos recursos disponíveis, tempo sendo um dos recursos.

No mundo acadêmico, a questão leva a um interessante, mas investigação da *otimização* of an algorithm. Na verdade, esta que

eficiência de um algoritmo, mas sobre a complexidade do problema e a quantidade mínima de esforço que *qualquer* algoritmo precisa para resolvê-lo? Para alguns problemas, a resposta a esta pergunta é com qualquer algoritmo que classifica uma matriz comparando os valores $n \log^2 n$ comparações para alguns arrays de tamanho n (consulte problemas aparentemente fáceis, como multiplicação de inteiros, ainda não tenho uma resposta final).

Outra questão importante da resolução de problemas algorítmicos é se todos os problemas podem ou não ser resolvidos por um algoritmo. Aqui sobre problemas que não têm solução, como encontrar raízes de uma equação quadrática com um discriminante negativo. Para tais indicando que o problema não tem solução é tudo o que podemos esperar de um algoritmo. Nem estamos falando de problemas formais. Mesmo alguns problemas inequívocos que devem ter uma resposta: “Indecidível”, ou seja, insolúvel por qualquer algoritmo. Um exemplo de um problema aparece na Seção 11.3. Felizmente, a grande maioria das computações práticas *pode* ser resolvida por um algoritmo.

Antes de sair desta seção, vamos ter certeza de que você não ficou equivocado - possivelmente causado pela natureza um tanto mecânica do diagrama da Figura 1.2 - que projetar um algoritmo é uma atividade nada mais longe da verdade: inventar (ou descobrir?) algoritmos é um processo criativo e gratificante. Este livro foi elaborado para convencer você disso. O caso.